EC 425 : Computer Graphics and Multimedia

QUIZ 2

Phong Shading using Ray Tracing

Implementation: 3 spheres, 1 light, checkered floor

Name: Solanki Kuldipkumar Kishorbhai
Roll number: 202351136
Course: EC 425 : Computer Graphics and Multimedia
Assignment: Quiz 2

---

1. What Was Implemented

A ray tracer that colours each pixel by casting a camera ray, finding the nearest hit among three spheres (plus a checkered ground plane), and evaluating Phong lighting at that point. Shadow rays darken occluded points. Mirror reflections are added with an iterative bounce loop (depth limited).

Table 1: Scene data

Entity	Stored values
Camera	eye position, look-at, FOV, resolution
Light (1)	position, intensity / colour
Sphere ($\times 3$)	centre, radius, albedo, k_a, k_d, k_s, n, k_r
Ground	$y = -1$ plane, procedural checker albedo

Pipeline per pixel:

1. Build primary ray through pixel (i, j).
2. Intersect ray with ground + 3 spheres; keep smallest $t > \varepsilon$.
3. At hit P: compute $\mathbf{N}$, $\mathbf{L}$, $\mathbf{V}$, $\mathbf{R}$.
4. Phong colour $= k_a I_A + k_d I_\ell \max(0, \mathbf{N} \cdot \mathbf{L}) + k_s I_\ell \max(0, \mathbf{R} \cdot \mathbf{V})^n$.
5. If reflective and bounce $<$ max depth, cast reflected ray and blend with weight k_r.

2. Core Pseudocode

```
procedure RenderImage():
    for each pixel (i, j):
        d <- primaryRayDirection(i, j)
        image[i,j] <- TraceRay(eye, d, bounce=0)
```

Listing 1: Render loop

```
function HitSphere(O, d, S):
    oc <- O - S.center
    b  <- dot(d, oc)
    c  <- dot(oc, oc) - S.radius^2
    disc <- b*b - c
    if disc < 0: return NONE
    t <- -b - sqrt(disc)
```

```
    if t < EPS: t <- -b + sqrt(disc)
    if t < EPS: return NONE
    return t

function FindNearest(O, d):
    bestT <- +inf; best <- NONE
    // ground plane + each of 3 spheres
    for each object in scene:
        t <- intersect(O, d, object)
        if t != NONE and t < bestT:
            bestT <- t; best <- object
    return (best, bestT)
```

Listing 2: Nearest hit among 3 spheres + ground

```
function EvaluatePhong(P, N, V, mat, light):
    amb <- mat.ka * ambient * mat.albedo
    L   <- normalize(light.pos - P)
    if shadowRayBlocked(P, L): return amb
    diff <- mat.kd * light.I * max(0, dot(N,L)) * mat.albedo
    R    <- 2*dot(N,L)*N - L
    spec <- mat.ks * light.I * max(0, dot(R,V))^mat.n
    return amb + diff + spec
```

Listing 3: Phong shade + shadow test

```
function TraceRay(O, d, bounce):
    (obj, t) <- FindNearest(O, d)
    if obj == NONE: return background

    P <- O + t*d
    N <- normalize(P - obj.center)   // or (0,1,0) on ground
    col <- EvaluatePhong(P, N, -d, obj, light)

    if bounce < MAX_BOUNCE and obj.kr > 0:
        rd  <- d - 2*dot(d,N)*N
        col <- col + obj.kr * TraceRay(P + EPS*N, rd, bounce+1)
    return saturate(col)
```

Listing 4: Iterative reflections

3. Rendered Output

Figures below are live renders from this implementation (3 spheres + checkered floor + 1 point light).

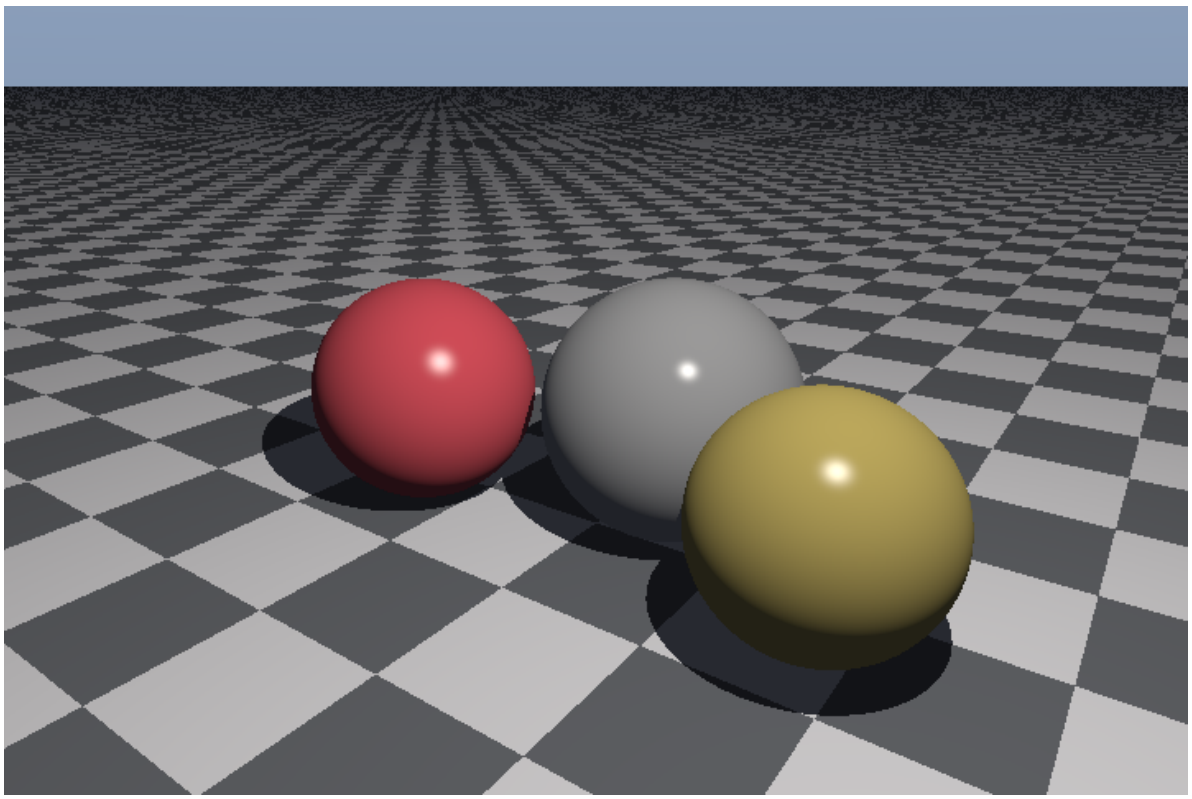


Figure 1: Phong only (ambient + diffuse + specular + shadows; reflections off).

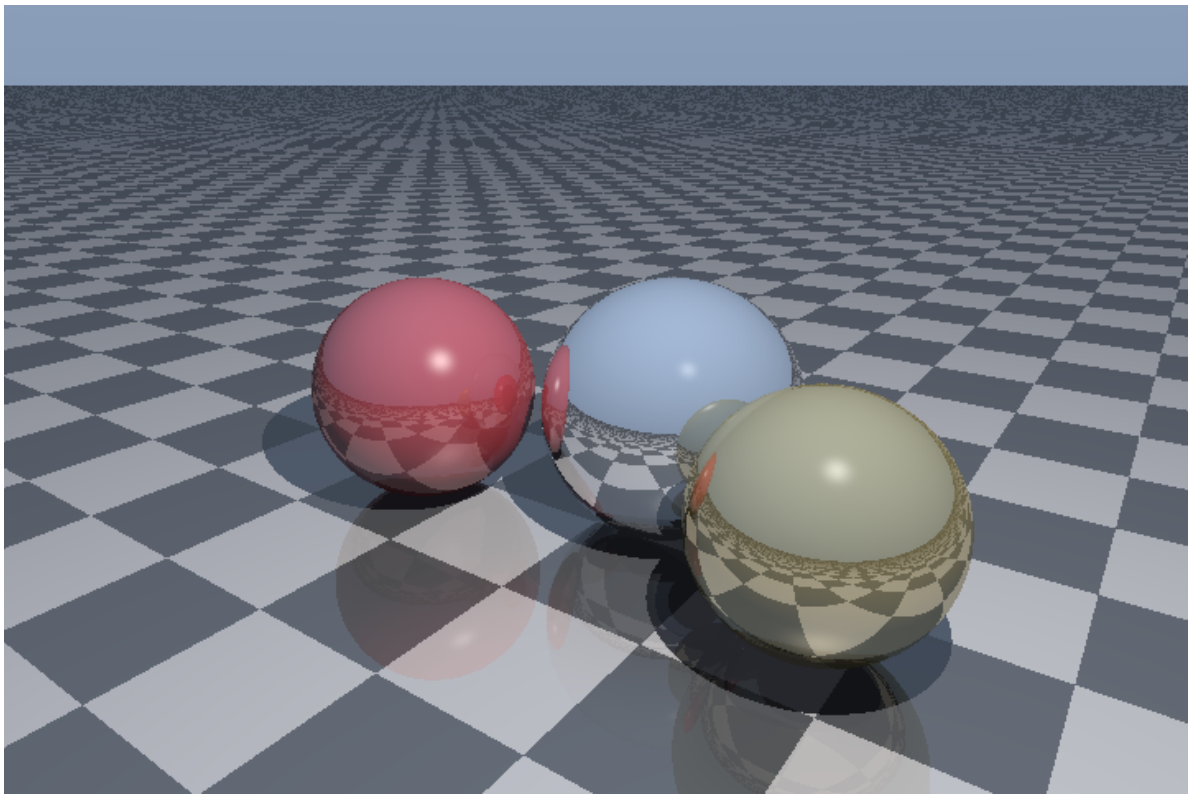


Figure 2: Phong + iterative reflections (max bounce depth = 4).

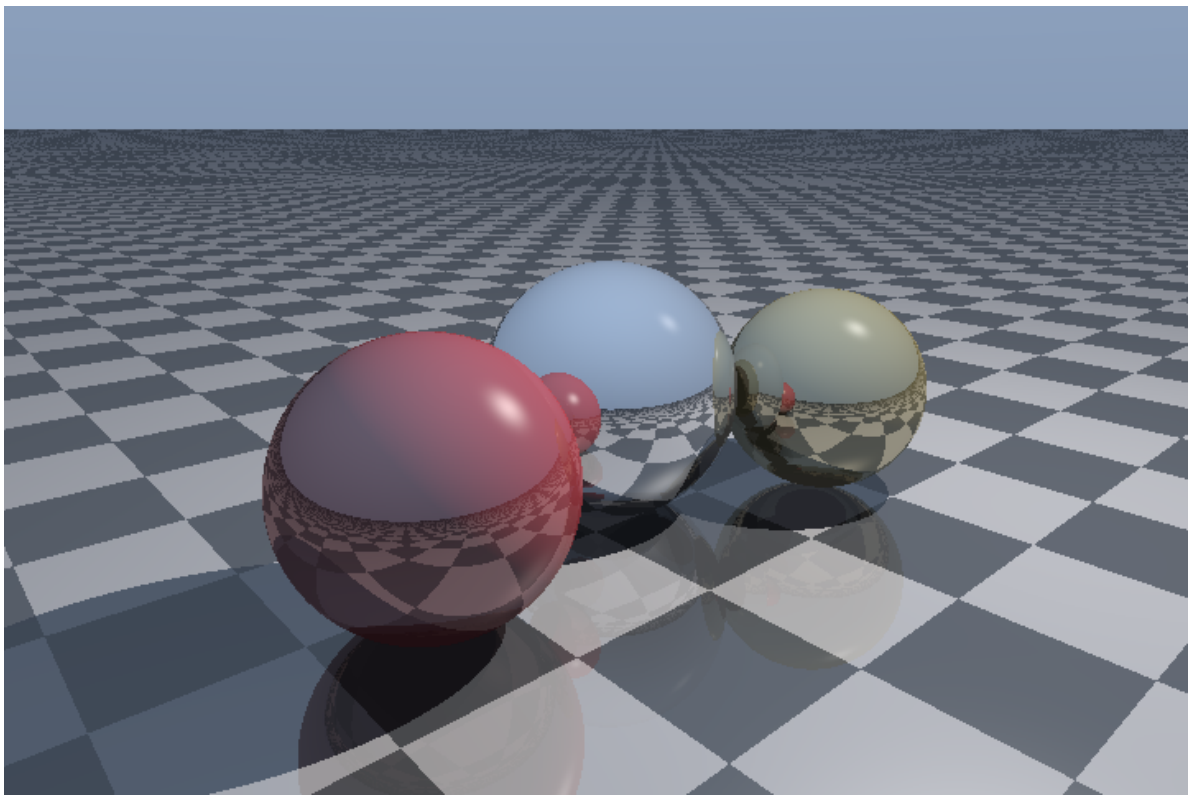


Figure 3: Same reflective scene from an alternate camera.

4. Brief Notes

- Sphere materials used: crimson (glossy), chrome (high k_r), brass (metallic).
- Shadow rays skip the emitter; an ε bias avoids self-intersection.
- Throughput is scaled by k_r each bounce and terminated early if negligible.

5. GitHub Repository

Source code, render outputs and this report:
`https://github.com/kuldipsolanki46/EC425_Quiz2_Phong_RayTracer`

A. Source Code

SDL3 + OpenGL 3.3 fragment-shader ray tracer for the same 3-sphere scene. Phong terms and bounce loop run on the GPU.

```
// EC425 Quiz 2 - iterative GPU ray tracer (SDL3 + OpenGL 3.3 + GLAD)
// Three spheres, checkered ground plane, one orbiting point light, shadows.

#include <SDL3/SDL.h>
#include <glad/glad.h>
#include <algorithm>
#include <cmath>
#include <cstddef>
#include <iostream>
#include <vector>

namespace {

```

```
constexpr const char* kVertSrc = R"GLSL(
#version 330 core
layout (location = 0) in vec2 pos;
out vec2 uv;
void main() {
    uv = pos * 0.5 + 0.5;
    gl_Position = vec4(pos, 0.0, 1.0);
}
)GLSL";

constexpr const char* kFragSrc = R"GLSL(
#version 330 core
in vec2 uv;
out vec4 outColor;

uniform vec2  uRes;
uniform float uTime;
uniform vec3  uEye;
uniform vec3  uLookAt;
uniform vec3  uLight;

const int   kMaxBounce = 4;
const float kInf = 1e20;

struct Ray  { vec3 o; vec3 d; };
struct Hit  {
    bool  valid;
    float t;
    vec3  p, n, albedo;
    float ks, shininess, kr;
};
struct Ball {
    vec3  c;
    float r;
    vec3  albedo;
    float ks, shininess, kr;
};

bool hitBall(Ray ray, Ball s, inout float tBest, inout Hit h) {
    vec3  oc = ray.o - s.c;
    float B  = dot(oc, ray.d);
    float C  = dot(oc, oc) - s.r * s.r;
    float D  = B*B - C;
    if (D <= 0.0) return false;
    float sd = sqrt(D);
    float t  = -B - sd;
    if (t <= 1e-3) t = -B + sd;
    if (t <= 1e-3 || t >= tBest) return false;
    tBest = t;
    h.valid = true; h.t = t;
    h.p = ray.o + t * ray.d;
    h.n = normalize(h.p - s.c);
    h.albedo = s.albedo;
    h.ks = s.ks; h.shininess = s.shininess; h.kr = s.kr;
    return true;
}

bool hitGround(Ray ray, float yPlane, inout float tBest, inout Hit h) {
    if (abs(ray.d.y) < 1e-5) return false;
    float t = (yPlane - ray.o.y) / ray.d.y;
    if (t <= 1e-3 || t >= tBest) return false;
    tBest = t;
    h.valid = true; h.t = t;
    h.p = ray.o + t * ray.d;
    h.n = vec3(0.0, 1.0, 0.0);
    vec2 cell = floor(h.p.xz * 0.8);
```

```
    bool dark = mod(cell.x + cell.y, 2.0) < 0.5;
    h.albedo = dark ? vec3(0.15, 0.16, 0.18) : vec3(0.82, 0.82, 0.86);
    h.ks = 0.35; h.shininess = 28.0; h.kr = 0.22;
    return true;
}

Hit castScene(Ray ray, Ball balls[3]) {
    Hit h; h.valid = false;
    float tBest = kInf;
    hitGround(ray, -1.0, tBest, h);
    for (int i = 0; i < 3; ++i) hitBall(ray, balls[i], tBest, h);
    return h;
}

bool occluded(vec3 p, vec3 Ldir, float Ldist, Ball balls[3]) {
    Ray sr; sr.o = p + Ldir * 0.003; sr.d = Ldir;
    for (int i = 0; i < 3; ++i) {
        vec3  oc = sr.o - balls[i].c;
        float B  = dot(oc, sr.d);
        float C  = dot(oc, oc) - balls[i].r * balls[i].r;
        float D  = B*B - C;
        if (D > 0.0) {
            float t = -B - sqrt(D);
            if (t > 1e-3 && t < Ldist) return true;
        }
    }
    return false;
}

Ray makePrimary(vec2 tex, vec3 eye, vec3 target, float fovY, float aspect) {
    vec3 f = normalize(target - eye);
    vec3 r = normalize(cross(f, vec3(0.0, 1.0, 0.0)));
    vec3 u = cross(r, f);
    float th = tan(radians(fovY) * 0.5);
    vec2 sc = (tex * 2.0 - 1.0) * vec2(aspect, 1.0);
    vec3 dir = normalize(f + sc.x * r * th + sc.y * u * th);
    return Ray(eye, dir);
}

void main() {
    float aspect = uRes.x / uRes.y;
    Ball balls[3];

    // crimson glossy
    balls[0] = Ball(vec3(-1.7, 0.05*sin(uTime*1.4), 0.3), 1.0,
                    vec3(0.88, 0.10, 0.14), 0.75, 56.0, 0.32);
    // near-mirror metal
    balls[1] = Ball(vec3(0.5, 0.05, -0.4), 1.1,
                    vec3(0.92, 0.93, 0.96), 1.0, 140.0, 0.80);
    // brass
    balls[2] = Ball(vec3(2.2, -0.05+0.12*cos(uTime*1.2), 0.6), 0.9,
                    vec3(0.90, 0.72, 0.20), 0.85, 72.0, 0.58);

    Ray ray = makePrimary(uv, uEye, uLookAt, 50.0, aspect);
    vec3 lightCol = vec3(1.0, 0.97, 0.92);
    vec3 ambCol   = vec3(0.06, 0.07, 0.10);

    vec3 color = vec3(0.0);
    vec3 weight = vec3(1.0);

    for (int b = 0; b < kMaxBounce; ++b) {
        Hit h = castScene(ray, balls);
        if (!h.valid) {
            float t = clamp(ray.d.y * 0.5 + 0.5, 0.0, 1.0);
            color += weight * mix(vec3(0.10, 0.12, 0.18),
                                  vec3(0.40, 0.55, 0.78), t);
```

```
            break;
        }

        vec3 toL = uLight - h.p;
        float distL = length(toL);
        vec3 L = toL / distL;
        vec3 V = -ray.d;
        bool inShade = occluded(h.p, L, distL, balls);

        vec3 ambient = ambCol * h.albedo;
        vec3 diffuse = vec3(0.0);
        vec3 specular = vec3(0.0);

        if (!inShade) {
            float ndl = max(dot(h.n, L), 0.0);
            diffuse = ndl * lightCol * h.albedo;
            vec3 R = reflect(-L, h.n);
            float rdv = max(dot(R, V), 0.0);
            specular = pow(rdv, h.shininess) * h.ks * lightCol;
        }

        color += weight * (1.0 - h.kr) * (ambient + diffuse);
        color += weight * specular;

        weight *= h.kr * mix(vec3(1.0), h.albedo, 0.45);
        if (max(weight.r, max(weight.g, weight.b)) < 0.012) break;

        ray.o = h.p + h.n * 0.003;
        ray.d = reflect(ray.d, h.n);
    }

    color = pow(color, vec3(1.0/2.2));
    outColor = vec4(color, 1.0);
}
)GLSL";

GLuint buildShader(GLenum type, const char* src) {
    GLuint sh = glCreateShader(type);
    glShaderSource(sh, 1, &src, nullptr);
    glCompileShader(sh);
    GLint ok = GL_FALSE;
    glGetShaderiv(sh, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        GLint len = 0;
        glGetShaderiv(sh, GL_INFO_LOG_LENGTH, &len);
        std::vector<char> log(std::max(len, 1));
        glGetShaderInfoLog(sh, len, nullptr, log.data());
        std::cerr << "compile error:\n" << log.data() << '\n';
        glDeleteShader(sh);
        return 0;
    }
    return sh;
}

GLuint linkProgram(const char* vs, const char* fs) {
    GLuint v = buildShader(GL_VERTEX_SHADER, vs);
    if (!v) return 0;
    GLuint f = buildShader(GL_FRAGMENT_SHADER, fs);
    if (!f) { glDeleteShader(v); return 0; }
    GLuint prog = glCreateProgram();
    glAttachShader(prog, v);
    glAttachShader(prog, f);
    glLinkProgram(prog);
    GLint ok = GL_FALSE;
    glGetProgramiv(prog, GL_LINK_STATUS, &ok);
    if (!ok) {
```

```
        GLint len = 0;
        glGetProgramiv(prog, GL_INFO_LOG_LENGTH, &len);
        std::vector<char> log(std::max(len, 1));
        glGetProgramInfoLog(prog, len, nullptr, log.data());
        std::cerr << "link error:\n" << log.data() << '\n';
        glDeleteProgram(prog);
        glDeleteShader(v);
        glDeleteShader(f);
        return 0;
    }
    glDeleteShader(v);
    glDeleteShader(f);
    return prog;
}

} // namespace

int main() {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init: " << SDL_GetError() << '\n';
        return 1;
    }
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 3);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 3);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);
    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);

    SDL_Window* win = SDL_CreateWindow(
        "EC425 Quiz2 - 3 Sphere Phong Ray Tracer", 960, 720,
        SDL_WINDOW_OPENGL | SDL_WINDOW_RESIZABLE);
    if (!win) {
        std::cerr << "window: " << SDL_GetError() << '\n';
        SDL_Quit();
        return 1;
    }
    SDL_GLContext ctx = SDL_GL_CreateContext(win);
    if (!ctx) {
        std::cerr << "context: " << SDL_GetError() << '\n';
        SDL_DestroyWindow(win);
        SDL_Quit();
        return 1;
    }
    if (!gladLoadGL(reinterpret_cast<GLADloadfunc>(SDL_GL_GetProcAddress))) {
        std::cerr << "GLAD failed\n";
        SDL_GL_DestroyContext(ctx);
        SDL_DestroyWindow(win);
        SDL_Quit();
        return 1;
    }
    SDL_GL_SetSwapInterval(1);

    GLuint prog = linkProgram(kVertSrc, kFragSrc);
    if (!prog) {
        SDL_GL_DestroyContext(ctx);
        SDL_DestroyWindow(win);
        SDL_Quit();
        return 1;
    }

    const float quad[] = {
        -1.f,-1.f,  1.f,-1.f,  1.f,1.f,
        -1.f,-1.f,  1.f,1.f,  -1.f,1.f
    };
    GLuint vao=0, vbo=0;
    glGenVertexArrays(1, &vao);
    glGenBuffers(1, &vbo);
```

```
    glBindVertexArray(vao);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(quad), quad, GL_STATIC_DRAW);
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2*sizeof(float), nullptr);
    glEnableVertexAttribArray(0);

    GLint locRes   = glGetUniformLocation(prog, "uRes");
    GLint locTime  = glGetUniformLocation(prog, "uTime");
    GLint locEye   = glGetUniformLocation(prog, "uEye");
    GLint locLook  = glGetUniformLocation(prog, "uLookAt");
    GLint locLight = glGetUniformLocation(prog, "uLight");

    float dist = 6.5f, yaw = 0.f, pitch = 0.32f;
    bool drag = false, anim = true;
    float mx = 0.f, my = 0.f;

    bool alive = true;
    while (alive) {
        SDL_Event ev;
        while (SDL_PollEvent(&ev)) {
            if (ev.type == SDL_EVENT_QUIT) alive = false;
            else if (ev.type == SDL_EVENT_KEY_DOWN) {
                if (ev.key.key == SDLK_ESCAPE) alive = false;
                if (ev.key.key == SDLK_SPACE) anim = !anim;
            } else if (ev.type == SDL_EVENT_MOUSE_BUTTON_DOWN &&
                       ev.button.button == SDL_BUTTON_LEFT) {
                drag = true; mx = ev.button.x; my = ev.button.y;
            } else if (ev.type == SDL_EVENT_MOUSE_BUTTON_UP &&
                       ev.button.button == SDL_BUTTON_LEFT) {
                drag = false;
            } else if (ev.type == SDL_EVENT_MOUSE_MOTION && drag) {
                yaw   += (ev.motion.x - mx) * 0.0055f;
                pitch  = std::clamp(pitch + (ev.motion.y - my)*0.0055f, 0.05f, 1.4f
                    );
                mx = ev.motion.x; my = ev.motion.y;
            }
        }

        int w=0, h=0;
        SDL_GetWindowSizeInPixels(win, &w, &h);
        if (w <= 0 || h <= 0) { SDL_Delay(16); continue; }

        glViewport(0, 0, w, h);
        glClear(GL_COLOR_BUFFER_BIT);

        float t = SDL_GetTicks() * 0.001f;
        float tAnim = anim ? t : 0.f;
        float ex = dist * std::cos(pitch) * std::sin(yaw);
        float ey = dist * std::sin(pitch);
        float ez = dist * std::cos(pitch) * std::cos(yaw);
        float lx = 3.8f * std::cos(t * 0.45f);
        float ly = 5.0f;
        float lz = 3.8f * std::sin(t * 0.45f) + 0.8f;

        glUseProgram(prog);
        glUniform2f(locRes, (float)w, (float)h);
        glUniform1f(locTime, tAnim);
        glUniform3f(locEye, ex, ey, ez);
        glUniform3f(locLook, 0.f, 0.15f, 0.f);
        glUniform3f(locLight, lx, ly, lz);

        glBindVertexArray(vao);
        glDrawArrays(GL_TRIANGLES, 0, 6);
        SDL_GL_SwapWindow(win);
    }

```

```
343     glDeleteBuffers(1, &vbo);
344     glDeleteVertexArrays(1, &vao);
345     glDeleteProgram(prog);
346     SDL_GL_DestroyContext(ctx);
347     SDL_DestroyWindow(win);
348     SDL_Quit();
349     return 0;
350 }
```

Listing 5: raytracer_phong.cpp